Instituto Politécnico do Cávado e do Ave
Escola Superior de Tecnologia

Engenharia Eletrotécnica e de Computadores

Trabalho Prático

Controlo, Navegação e Processamento de Imagem
Robô Móvel

Tiago Oliveira 25547

Ivo Azevedo 15526

Bruno Fernandes 15455

Grupo 6

Fevereiro de 2026

Resumo

Este trabalho descreve a criação e o desenvolvimento de um sistema de controlo e navegação para um robô móvel, que se pode deslocar em um ambiente delimitado com várias zonas proibidas e sinais de trânsito.

O sistema é constituído por técnicas de processamento de imagem, cinemática do robô para a sua movimentação, planeamento de trajetórias e reconhecimento de sinais para executar uma trajetória completa de forma autónoma.

O sistema desenvolvido contém a captura de uma imagem superior da pista para realizar o mapeamento inicial do ambiente, com a identificação da posição das bases coloridas, os limites da pista e a localização do robô. Através do processamento do mapa, o sistema converte pixéis em distância real por pixéis, o que permite a navegação do robô no mundo.

O movimento do robô foi implementado seguindo os modelos de cinemática direta e inversa, o que garantiu o controlo da velocidade e movimentos angulares do robô com a máxima precisão possível. O planeamento de trajetórias foi desenvolvido tendo em conta algoritmos para evitar zonas proibidas. A sequência de visita às bases é definida exclusivamente em software.

Quando o robô se encontra devidamente posicionado na base, o sistema captura (via câmara IP) a imagem do sinal posicionado na base e identifica-o. Seguidamente executa a ação correspondente (virar à esquerda, virar à direita ou parar). O ciclo do sistema termina quando o robô chega ao sinal STOP.

O trabalho mostra a aplicação prática de conceitos de robótica móvel e processamento de imagem.

Índice

1 Introdução 6
2 Fundamentos Teóricos 7
2.1 Cinemática 7
2.1.1 Cinemática Direta 7
2.1.2 Cinemática Inversa 7
2.2 Visão Computacional 8
2.2.1 Processamento de Imagem 8
2.2.2 Calibração Câmara-Mundo 8
2.3 Localização e Mapeamento 9
2.3.1 Odometria 9
2.3.2 Relocalização por Visão 9
2.4 Planeamento de Trajetórias 9
3 Arquitetura do Sistema 10
3.1 Visão Geral do Sistema 10
3.2 Hardware Utilizado 10
3.2.1 Robô Móvel Lego Mindstorms 10
3.2.2 Sistema de Visão Superior 11
3.2.3 Computador de Controlo 11
3.3 Arquitetura de Software 11
3.3.1 Estrutura Modular 11
3.3.2 Linguagens e Bibliotecas 12
3.3.3 Fluxo de Informação 13
3.4 Decisões de Projeto 14
3.4.1 Processamento Centralizado vs. Distribuído 14
3.4.2 Representação do Mapa 14
3.4.3 Estratégia de Relocalização 14
4 Implementação 15
4.1 Mapeamento do Ambiente 15
4.1.1 Captura da imagem superior 15
4.1.2 Calibração Imagem-Mundo 15
4.1.3 Deteção das bases 16
4.1.4 Zonas Proibidas 16
4.1.5 Posição inicial do Robô 17

4.2 Controlo de movimento 17
4.2.1 Implementação da cinemática direta 17
4.2.2 Implementação da cinemática inversa 17
4.2.3 Controlo dos motores 18
4.3 Planeamento das Trajetórias 18
4.3.1 Algoritmo de planeamento 18
4.3.2 Zonas Proibidas 18
4.3.3 Sequência de visita às bases 18
4.4 Reconhecimento de Sinais 19
4.4.1 Captura da imagem vista pelo robô 19
4.4.2 Pré-Processamento 19
4.4.3 Classificação dos sinais 19
4.4.4 Execução 19
5 Resultados e Discussão 20
5.1 Mapeamento do Ambiente 20
5.2 Planeamento e Execução de Trajetórias 21
5.3 Correção de Posição por Relocalização Visual 22
5.4 Reconhecimento de Sinais de Trânsito 23
5.5 Desempenho Global do Sistema 24
5.6 Análise Crítica e Limitações 25
6 Conclusão 26

Índice de Figuras

Figura 1 – Diagrama de Blocos do Sistema 13
Figura 2 – Imagem Capturada pela câmara Top-Down 15
Figura 3 – Processamento das Bases da Imagem Capturada 16
Figura 4 – Processamento do Mapa Total da Imagem Capturada 16
Figura 5 – Processamento do Mapa de Zona Livre da Imagem Capturada 17
Figura 6 – Processamento do Robô da Imagem Capturada 17
Figura 7 - Trajetória completa executada pelo robô autonomamente 21
Figura 8 - Correção da posição real do robô através de visão computacional 22
Figura 9 - Reconhecimento de Sinais de Trânsito 23

Índice de Equações

Equação 1 - Velocidade Angular 7
Equação 2 - Velocidade Linear 7
Equação 3 - Velocidade da Roda Direita 7
Equação 4 – Velocidade da Roda Esquerda 7
Equação 5 – Conversão Pixéis em Metros 8
Equação 6 – Conversão Metros para Pixéis 8
Equação 7 – Conversão de milímetros para pixéis (real para virtual) 15

1 Introdução

Neste trabalho prático, foi desenvolvido um sistema de controlo, navegação e visão computacional para um robô móvel Lego Mindstorms EV3, capaz de operar de forma autónoma num cenário montado, mas sujeito a mudanças na configuração.

O problema proposto consistiu na navegação do robô no interior de uma arena delimitada, onde existem várias bases representadas por folhas coloridas. Cada base define uma zona reservada em que o robô não pode circular, sendo necessário aproximar-se de cada base sem passar por cima da respetiva folha. Adicionalmente, sobre cada base é colocado um sinal de trânsito (obrigatório à esquerda, obrigatório à direita e STOP), que deve ser detetado e reconhecido através de visão por computador, o que desencadeia diferentes comportamentos do robô, dependendo do sinal lido.

No início da execução, é capturada uma fotografia "top-down" da pista, a partir da qual o sistema deve mapear o ambiente, identificando a posição inicial do robô, a posição das bases, as zonas proibidas e os limites da pista, incluindo também a etapa de calibração para converter coordenadas da imagem em distâncias reais. Durante a execução, pode ser utilizada uma nova fotografia apenas para relocalização/correção da posição, mantendo-se o mapa das bases fixo. A navegação decorre de forma autónoma entre as bases, com a ordem definida em software, repetindo-se o ciclo até ao reconhecimento do sinal STOP, momento em que o robô deve parar e o programa terminar de forma controlada.

O movimento do robô foi implementado com um controlador ponto-a-ponto (*Drive-to-Point*), que conduz o robô entre *waypoints* sucessivos, convertendo (v, ω) em velocidades das rodas.

Ao longo deste relatório são apresentados os principais módulos do sistema:

- Mapeamento e calibração;
- Planeamento de trajetórias;
- Controlo de movimento;
- Reconhecimento de sinais.

Posteriormente apresentamos as decisões de implementação e no final os resultados obtidos em testes na arena.

2 Fundamentos Teóricos

Neste capítulo vamos fazer uma abordagem teórica aos seguintes temas:

- Cinemáticas;
- Processamento de imagem;
- Planeamento de trajetórias.

Estes foram os principais tópicos para o desenvolvimento deste trabalho.

2.1 Cinemática

2.1.1 Cinemática Direta

A cinemática direta permite determinar a variação da posição do robô conhecendo as velocidades das rodas. A integração das velocidades ao longo do tempo fornece a estimativa da posição e orientação.

$$v = \frac{wheelRadius}{2} * (w_L + w_R)$$

Equação 2 - Velocidade Linear

$$w = \frac{wheelRadius}{wheelBase} * (w_L - w_R)$$

Equação 1 - Velocidade Angular

Este modelo é fundamental para a odometria, pois estima a posição do robô ao longo da sua trajetória através da integração numérica destas equações diferenciais.

2.1.2 Cinemática Inversa

A cinemática inversa determina as velocidades necessárias em cada roda para obter a velocidade linear e angular desejadas.

$$w_L = \frac{(\frac{v - (w * wheelBase)}{2})}{wheelRadius}$$

Equação 4 – Velocidade da Roda Esquerda

$$w_R = \frac{(\frac{v + (w * wheelBase)}{2})}{wheelRadius}$$

Equação 3 - Velocidade da Roda Direita

Este modelo é essencial para controlar o robô, permitindo converter velocidades lineares e velocidades angulares em velocidades específicas para cada motor.

2.2 Visão Computacional

2.2.1 Processamento de Imagem

O processamento de imagem é um dos pilares do trabalho, responsável pela perceção do sistema. As principais operações incluem:

- **Conversão de espaços de cor**: transformação de RGB para HSV, espaço este mais adequado à deteção de cores nos ambientes com iluminação variável;
- **Segmentação por cor**: separação de cores com base em intervalos de valores nos diferentes canais de cor;
- **Operações morfológicas**: aplicação de erosões e dilatações para eliminar ruído e aumentar certos objetos necessários;
- **Deteção de contornos**: identificação de limites, permitindo extrair a forma e posição de objetos.

2.2.2 Calibração Câmara-Mundo

A calibração estabelece a correspondência entre pixéis e metros. Recorrendo às bases que têm as dimensões conhecidas, é possível converter distâncias medidas na imagem em distâncias físicas na pista.

$$x_{Real} = (x_{Pixel} - x_{Origem}) * metrosPorPixel$$

$$y_{Real} = -(y_{Pixel} - y_{Origem}) * metrosPorPixel$$

Equação 5 – Conversão Pixéis em Metros

$$x_{Pixéis} = \frac{(x_{Origem} + x_{Metros})}{metrosPorPixel}$$

$$y_{Pixéis} = \frac{(y_{Origem} - y_{Metros})}{metrosPorPixel}$$

Equação 6 – Conversão Metros para Pixéis

(x_{Origem}, y_{Origem}) representam a origem do sistema.

2.3 Localização e Mapeamento

2.3.1 Odometria

A odometria consiste num método simples de localização, que se baseia na integração dos movimentos do robô ao longo do tempo. No entanto, apresenta acumulação de erros devido ao deslizamento das rodas, irregularidades da arena e imperfeições na calibração dos motores.

2.3.2 Relocalização por Visão

Para mitigar o erro acumulado da odometria, pode ser usada uma imagem *top-down* para voltar a estimar a posição e orientação do robô e corrigir a posição antes de continuar a navegação.

2.4 Planeamento de Trajetórias

O planeamento de trajetórias adotado baseia-se numa representação da arena através de uma máscara binária de ocupação (células livres e ocupadas). A partir da segmentação do mapa, é construído o espaço navegável e, para garantir as regras de navegação, as bases são dilatadas para que o robô tenha uma margem de segurança quando estiver a chegar às bases ou tiver de as contornar.

Depois, o caminho global é obtido calculando um campo de distâncias geodésicas sobre as células livres: para cada célula livre estima-se o custo mínimo para a atingir a partir do ponto inicial, respeitando obstáculos. Com esse campo, a trajetória é reconstruída por backtracking: a partir do alvo escolhem-se sucessivamente vizinhos com custo decrescente até chegar à origem, resultando num caminho válido no espaço livre.

Como o caminho em grelha tende a ter muitos pontos e “zig-zags”, aplica-se uma simplificação por linha de visão, removendo pontos intermédios sempre que dois pontos consecutivos possam ser ligados por um segmento inteiramente livre.

Por fim, o caminho é convertido num conjunto de waypoints para execução pelo controlador ponto-a-ponto do robô.

3 Arquitetura do Sistema

O sistema desenvolvido integra três blocos principais:

- Robô móvel diferencial LEGO Mindstorms EV3;
- Sistema de visão (uma câmara superior para mapear a arena e uma câmara no robô para a leitura dos sinais);
- Computador onde corre uma aplicação em MATLAB (App Designer).

3.1 Visão Geral do Sistema

O sistema desenvolvido pode ser decomposto em dois subsistemas principais: o subsistema de planeamento e controlo, e o subsistema de atuação/odometria no EV3

A comunicação com o EV3 é via USB, enquanto a aquisição de imagem é feita via câmaras IP na rede.

O fluxo de execução global do sistema segue a seguinte sequência:

1. **Fase de inicialização**: captura da imagem superior da pista, mapeamento do ambiente e calibração de *thresholds*;

2. **Fase de navegação**: ciclo iterativo de planeamento de trajetória, execução do movimento e relocalização (quando necessário);

3. **Fase de reconhecimento**: ao atingir cada base, captura e classificação do sinal de trânsito;

4. **Fase de terminal**: reconhecimento do sinal STOP e imobilização do robô.

3.2 Hardware Utilizado

3.2.1 Robô Móvel Lego Mindstorms

O robô é um sistema diferencial de duas rodas integrados nos motores. A nossa interação com o robô é feita via USB.

O modelo cinemático usa parâmetros físicos definidos no software, como por exemplo definimos o raio das rodas e a distância do eixo entre cada roda.

Estes parâmetros suportam a conversão entre velocidades desejadas (linear e angular) e velocidades de cada motor, bem como a atualização de odometria através dos incrementos dos *encoders*.

3.2.2 Sistema de Visão Superior

O sistema de mapeamento requer uma câmara posicionada acima da pista, com o eixo ótico perpendicular ao plano de navegação.

Esta câmara fornece a imagem necessária para:

- Identificar a arena;
- Detetar as bases;
- Gerar o mapa navegável e definir as zonas proibidas;
- Estimar a posição inicial do robô.

3.2.3 Computador de Controlo

O processamento de imagem de alto nível e o planeamento de trajetórias são executados num computador externo.

O computador de controlo executa a aplicação em MATLAB App Designer e atua como "cérebro" do sistema. Este, é responsável por:

- Adquirir e processar imagens;
- Executar planeamento e controlo em tempo real;
- Comunicar com o EV3 via USB para controlar os motores;
- Apresentar informações relevantes ao utilizador.

3.3 Arquitetura de Software

3.3.1 Estrutura Modular

O sistema foi desenvolvido em MATLAB App Designer. Apesar de não existirem ficheiros ".m" externos, o código mantém uma organização modular ao nível de métodos, com separação funcional por responsabilidades.

Os principais módulos lógicos são:

- **Aquisição e Visão (Câmaras e segmentação):** métodos responsáveis por ligar às câmaras, obter frames/snapshots e aplicar segmentação por cor em HSV com *thresholds* calibrados.
- **Mapeamento do ambiente:** métodos que processam a imagem *top-down* para extrair a arena, identificar bases e a posição do robô, calcular a escala e construir uma representação navegável.
- **Planeamento de trajetórias:** métodos que, a partir do mapa navegável, calculam um conjunto de *waypoints* entre a posição atual e o *waypoint* mais próximo da base alvo, garantindo que o percurso decorre em zonas livres e que a trajetória é adequada à execução.
- **Navegação e controlo de movimento:** métodos que executam a trajetória em dois níveis:
    - Em alto nível é apenas passado uma lista de *waypoints* para o robô se deslocar entre eles;
    - Em baixo nível, é utilizado um controlador para conduzir o robô até um ponto com rotação/avanço, convertendo comandos (v, ω) em velocidades de rodas via cinemática diferencial e aplicando limites/rampas de velocidade.
- **Localização:** métodos que atualizam a posição do robô com base em fotos captadas no início e durante o percurso, permitindo localizar o robô e se necessário corrigir a sua posição de forma reduzir o erro acumulado da odometria devido às múltiplas deslocações.
- **Coordenação do percurso:** método que prepara o ciclo completo: inicialização, planeamento até cada base, execução do movimento, deteção do sinal e execução da manobra/decisão.

3.3.2 Linguagens e Bibliotecas

A linguagem de programação utilizada para o desenvolvimento do trabalho foi o MATLAB.

Já as bibliotecas utilizadas foram:

- **LEGO MINDSTORMS EV3** - Para que fosse possível interagir com o robô;
- **IP CAMERA** - Para estabelecer ligação com as camaras dos telemóveis.

3.3.3 Fluxo de Informação

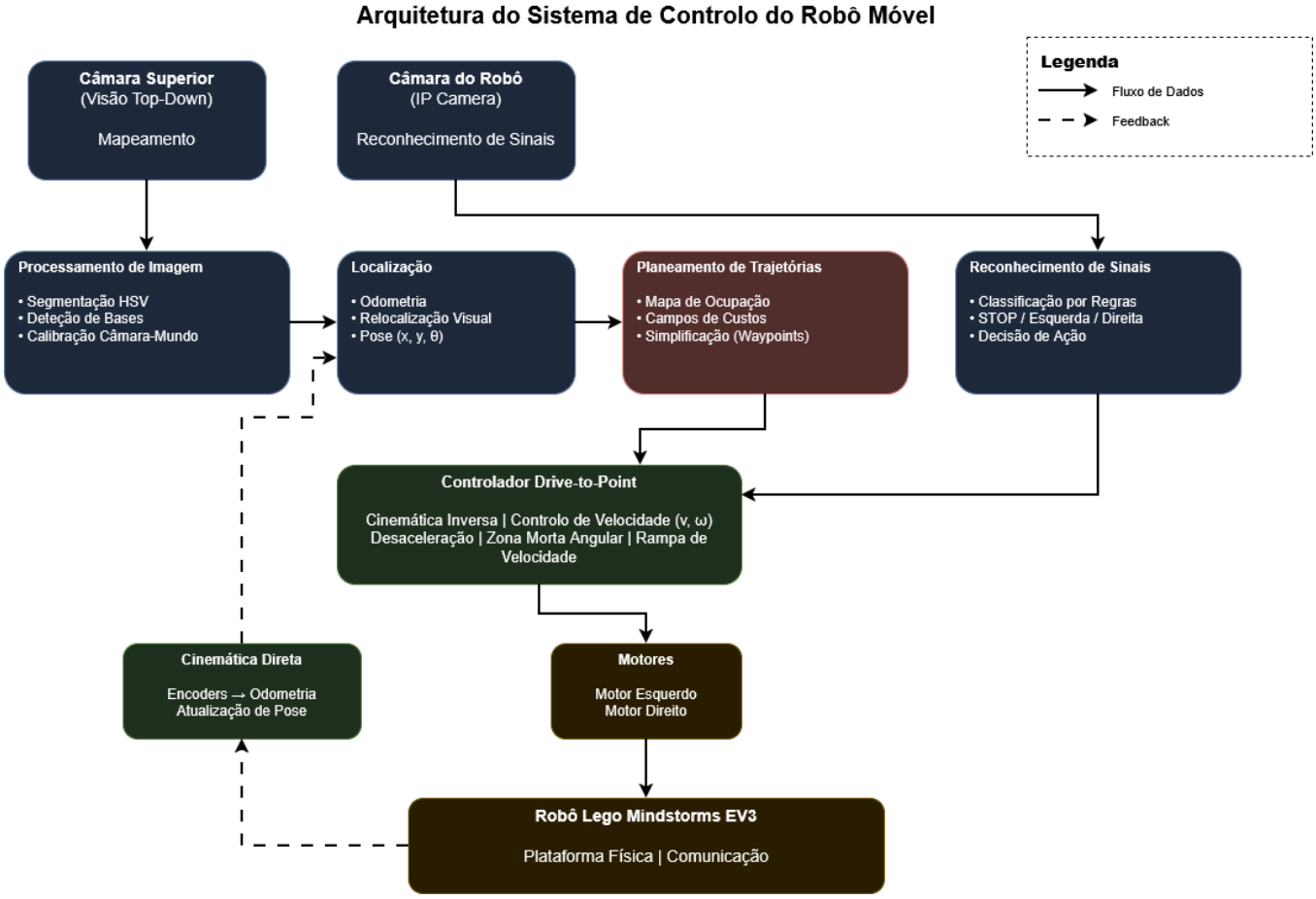


Figura 1 – Diagrama de Blocos do Sistema

O fluxo de informação entre módulos segue, de forma geral, a sequência:

- **Aquisição/Visão** captura imagens (*top-down* e/ou sinal) e extrai máscaras/medidas relevantes.
- **Mapeamento** gera/atualiza o mapa navegável e os pontos de referência (bases/*waypoints*).
- **Localização** define a posição inicial e atualiza-a continuamente via odometria (com possibilidade de correção visual).
- **Coordenação do percurso** define a base objetivo e chama o **Planeamento** para obter uma trajetória.
- **Navegação/Controlo** executa essa trajetória convertendo-a em comandos para o robô, atualizando a posição ao longo do movimento.
- Ao atingir a base, o módulo de **Visão** processa a imagem do sinal, classifica-o e a **Coordenação** decide a ação (contornar/seguir/parar).
- O ciclo repete-se até ser detetado o sinal **STOP**.

3.4 Decisões de Projeto

3.4.1 Processamento Centralizado vs. Distribuído

Optou-se por uma arquitetura centralizada, com o processamento pesado executado no computador externo. Esta decisão justifica-se pelas limitações computacionais do brick do Lego Mindstorms, que não possui capacidade para executar algoritmos complexos de visão e planeamento em tempo real. O EV3 fica apenas responsável pela execução dos comandos de movimento e pela disponibilização de odometria (*encoders*), enquanto a aquisição e processamento das imagens (mapa e sinais) são realizados no computador através de câmaras IP.

3.4.2 Representação do Mapa

O ambiente é representado por uma máscara binária de ocupação (zonas livres e zonas ocupadas) obtida por segmentação da imagem *top-down*. As bases são integradas como regiões ocupadas (com margem de segurança), e o planeamento opera exclusivamente sobre a zona livre.

3.4.3 Estratégia de Relocalização

A correção visual é executada em momentos chave, tipicamente após atingir o *waypoint* de aproximação da base e antes da leitura do sinal, para aumentar a repetibilidade do alinhamento e da aproximação. Caso a deteção falhe, o sistema mantém a pose estimada por odometria e prossegue a execução.

4 Implementação

Esta secção descreve como o sistema foi implementado, desde o mapeamento da arena até ao reconhecimento de sinais, destacando os principais módulos, o fluxo de dados e as decisões técnicas tomadas para garantir robustez e repetibilidade.

4.1 Mapeamento do Ambiente

4.1.1 Captura da imagem superior

O mapeamento inicia-se com a captura de uma imagem *top-down* da arena através de uma câmara IP posicionada perpendicularmente ao plano de navegação. Esta imagem serve de base para detetar o tabuleiro (zona válida), as bases coloridas e os marcadores do robô. A APP trabalha diretamente sobre a imagem RGB capturada, convertendo-a depois para HSV para permitir segmentação por cor mais robusta a variações de iluminação.

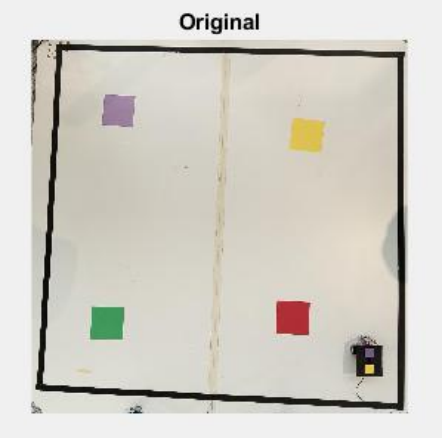


Figura 2 – Imagem Capturada pela câmara Top-Down

4.1.2 Calibração Imagem-Mundo

Após o processamento inicial, é estabelecida a conversão entre pixéis e unidades reais. A escala é calculada a partir da base vermelha, detetando a sua *BoundingBox* e assumindo-se dimensão real conhecida (200 mm). Assim, obtém-se:

$$mmPerPixel = 200 \,/\, média(largura_px, altura_px)$$

Equação 7 – Conversão de milímetros para pixéis (real para virtual)

Além disso, é definida uma origem em pixéis com base na geometria do tabuleiro. A orientação inicial é calculada pelo vetor entre os dois marcadores do robô (lilás e amarelo), considerando a inversão do eixo Y típica das imagens (em imagem o Y cresce para baixo).

4.1.3 Deteção das bases

A deteção das bases é feita por segmentação em HSV usando thresholds calibrados para cada cor. Para cada base, é criada uma máscara binária e aplicam-se operações morfológicas (remoção de ruído e preenchimento) para garantir regiões sólidas e estáveis.

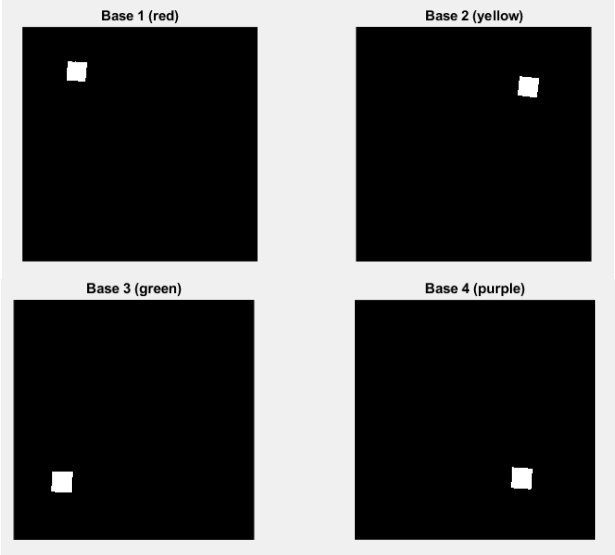


Figura 3 – Processamento das Bases da Imagem Capturada

Em paralelo, o tabuleiro é detetado pela componente branca, sendo também limpo com operações morfológicas para obter uma máscara consistente da área navegável.

Figura 4 – Processamento do Mapa Total da Imagem Capturada

4.1.4 Zonas Proibidas

As bases coloridas são tratadas como obstáculos. Para garantir uma margem de segurança equivalente ao volume do robô e reduzir o risco de calcar bases, cada base é dilatada com um elemento estruturante alargado.

A máscara de navegação final (mapa binário) é então construída como:

$$zona\ livre\ =\ tabuleiro\ -\ (bases\ dilatadas)$$

Ou seja, o robô apenas planeia e circula nas células livres do tabuleiro que não pertencem às zonas proibidas.

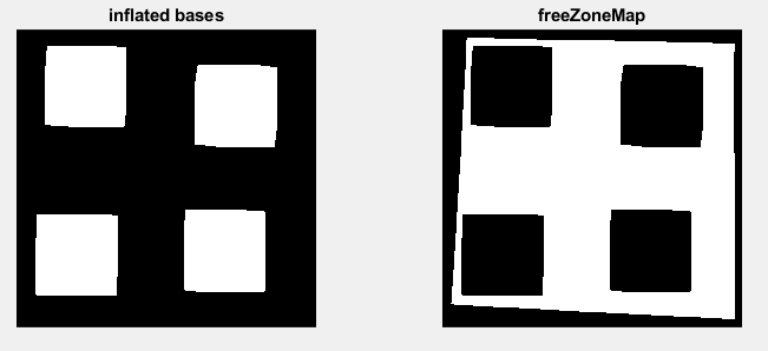


Figura 5 – Processamento do Mapa de Zona Livre da Imagem Capturada

4.1.5 Posição inicial do Robô

O robô é detetado na imagem através de dois marcadores coloridos (lilás e amarelo) sobre fundo preto. A posição inicial é obtida pelo centroide do marcador principal (lilás) e a orientação inicial é calculada pelo ângulo do vetor que liga os dois marcadores.

Figura 6 – Processamento do Robô da Imagem Capturada

4.2 Controlo de movimento

4.2.1 Implementação da cinemática direta

A cinemática direta é aplicada na odometria, a cada ciclo são lidos os *encoders*, calculados os incrementos de rotação das rodas e convertidos em deslocamento linear e variação angular.

A integração é feita considerando movimento por arco, melhorando a consistência da posição (x, y, θ) ao longo do tempo.

4.2.2 Implementação da cinemática inversa

A cinemática inversa converte os comandos de velocidade do robô (v, ω) em velocidades para cada motor, usando os parâmetros físicos (raio da roda e distância entre rodas). Isto permite que o controlo seja desenhado no espaço do robô e executado no espaço dos atuadores.

4.2.3 Controlo dos motores

O movimento é realizado por um controlador *Drive-To-Point*, que conduz o robô por uma sequência de *waypoints*. Para cada *waypoint*, calcula-se o erro de distância e o erro angular, ajustando v e ω. Para robustez e repetibilidade, a implementação inclui:

- limitação/saturação de velocidades,
- desaceleração ao aproximar do objetivo,
- zona morta angular para reduzir oscilações,
- rampa de velocidade para evitar variações bruscas,
- modo de rotação no local quando é necessário alinhar antes de avançar (útil na aproximação e leitura de sinal).

4.3 Planeamento das Trajetórias

4.3.1 Algoritmo de planeamento

A implementação do planeamento segue a metodologia descrita no subcapítulo 2.4. Em termos práticos, o sistema trabalha sobre um mapa binário de ocupação (livre/ocupado) gerado a partir do processamento da imagem *top-down*. O percurso é obtido através do cálculo de um campo de custos no espaço livre e reconstrução do caminho até ao objetivo, sendo depois aplicada uma simplificação por linha de visão para reduzir segmentos redundantes e produzir uma lista final de waypoints. Estes waypoints são então fornecidos ao controlador Drive-to-Point para execução.

4.3.2 Zonas Proibidas

No planeamento, as zonas proibidas já se encontram incorporadas no mapa binário como células ocupadas (após dilatação). Assim, qualquer trajetória gerada é automaticamente constrangida ao espaço livre, aumentando a tolerância a pequenos erros de execução e odometria.

4.3.3 Sequência de visita às bases

A sequência de bases é definida em software. Para cada base, o sistema seleciona automaticamente o *waypoint* de aproximação mais adequado, tipicamente o mais próximo da

posição atual, reduzindo trajetos desnecessários. Após a aproximação, a navegação integra também a lógica de decisão do sinal.

4.4 Reconhecimento de Sinais

4.4.1 Captura da imagem vista pelo robô

Quando o robô chega à base, é capturada uma imagem do sinal através de uma câmara IP dedicada. Para melhorar fiabilidade, a captura ocorre com o robô parado; se a leitura falhar, o sistema pode realizar pequenas rotações no local para obter uma vista mais frontal antes de repetir a tentativa.

4.4.2 Pré-Processamento

A imagem do sinal é convertida para HSV e são aplicados *thresholds* para segmentar as cores relevantes (vermelho, azul e branco). Em seguida são usadas operações morfológicas para reduzir ruído, consolidar regiões e preencher falhas, obtendo máscaras limpas para a decisão.

4.4.3 Classificação dos sinais

A classificação é feita por regras simples:

- **STOP**: presença de região vermelha dominante e elementos brancos no interior da região vermelha;
- **ESQUERDA/DIREITA**: presença de região azul dominante; a direção é inferida pela posição do branco (seta) relativamente ao centro do sinal (ex.: centroide do branco deslocado à direita → "DIREITA", caso contrário → "ESQUERDA").

4.4.4 Execução

Depois de classificar o sinal:

- **ESQUERDA/DIREITA**: o robô executa a manobra de contorno correspondente (mantendo-se fora da zona proibida) e segue para a próxima base;
- **STOP**: o robô para e o programa termina.

5 Resultados e Discussão

Esta secção apresenta e analisa os resultados obtidos durante os testes do sistema desenvolvido. São discutidos o desempenho de cada módulo e apresentados resultados reais que demonstram o funcionamento do sistema nas diferentes fases de execução.

5.1 Mapeamento do Ambiente

O processo de mapeamento automático demonstrou ser eficiente e confiável. Como demonstrado na secção de Implementação, o sistema foi capaz de identificar corretamente todas as bases coloridas, detetar os limites do tabuleiro e localizar a posição inicial do robô através dos marcadores coloridos (lilás e amarelo).

A calibração baseada na dimensão conhecida da base vermelha (200 mm) resultou em uma conversão pixel-para-milímetro consistente, com um erro mínimo quando comparado com medições manuais da pista. Essa precisão foi essencial para garantir que os caminhos planeados correspondessem às distâncias reais a serem percorridas.

As áreas navegáveis foram definidas corretamente pelo mapa de zonas livres, que foi gerado após a inflação das bases coloridas. A operação de dilatação, que aplicou uma margem de segurança, assegura que o robô permaneça afastado das áreas restritas, sem qualquer registo de invasão durante os testes realizados.

5.2 Planeamento e Execução de Trajetórias

O sistema de planeamento gerou consistentemente trajetórias válidas que evitavam as zonas proibidas.

A Figura seguinte ilustra um exemplo de trajetória completa executada pelo robô para visitar uma sequência de bases definidas em software.

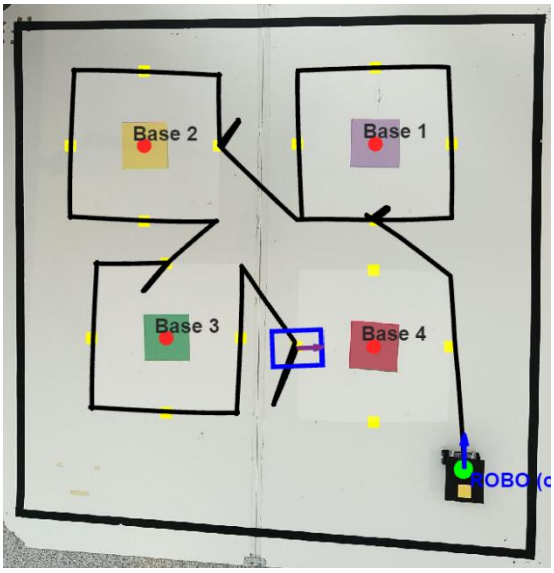


Figura 7 - Trajetória completa executada pelo robô autonomamente

Como pode ser visto, o robô seguiu um trajeto que evitou todas as áreas restritas, aproximando-se de cada base sem passar por cima das folhas coloridas. A simplificação por linha de visão reduziu o número de waypoints intermédios em aproximadamente 40-50%, tornando os percursos mais diretos e eficazes.

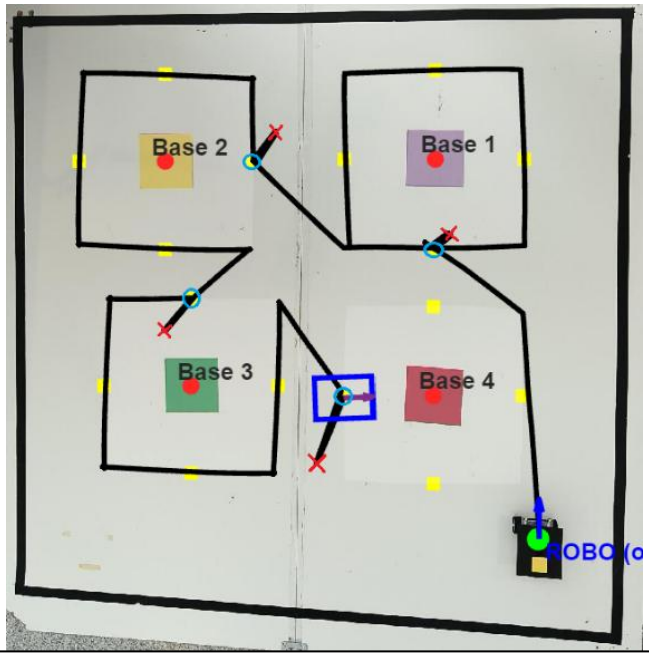
O controlador Drive-to-Point apresentou um desempenho satisfatório na condução do robô entre waypoints. Os mecanismos de desaceleração gradual e a rampa de velocidade resultaram em movimentos suaves e previsíveis, com o robô a atingir os waypoints com uma precisão média de aproximadamente 20-30 mm.

5.3 Correção de Posição por Relocalização Visual

Um dos maiores desafios foi a acumulação de erro odométrico perante longas trajetórias. Em trajetos que incluíam a passagem por todas as bases, o desvio entre a posição estimada e a posição real chegou a atingir valores significativos, considerando o tamanho da pista, antes da recolocação visual.

A relocalização visual, utilizada antes da aproximação final a cada base, demonstrou ser eficiente na correção desses erros acumulados.

A Figura seguinte apresenta um exemplo comparativo entre a posição estimada por odometria e a posição real do robô, tanto antes quanto depois da utilização da relocalização.



Legenda:

X – Posição Real do Robô dada pela snapshot do telemóvel

O – Posição Corrigida do Robô após a correção automática

Figura 8 - Correção da posição real do robô através de visão computacional

Após cada relocalização, o erro de posicionamento foi minimizado, tornando-se adequado para a aproximação e leitura dos sinais. Esse método foi eficaz em situações com trajetórias longas ou que envolviam múltiplas rotações, onde a acumulação de erro seria mais significativa.

5.4 Reconhecimento de Sinais de Trânsito

Quando o robô estava corretamente posicionado em relação ao sinal, o sistema de reconhecimento de sinais, que utiliza processamento de imagem no espaço HSV e classificação por regras, demonstrou uma alta taxa de sucesso. A próxima figura mostra exemplos de reconhecimento dos três tipos de sinais que foram implementados.

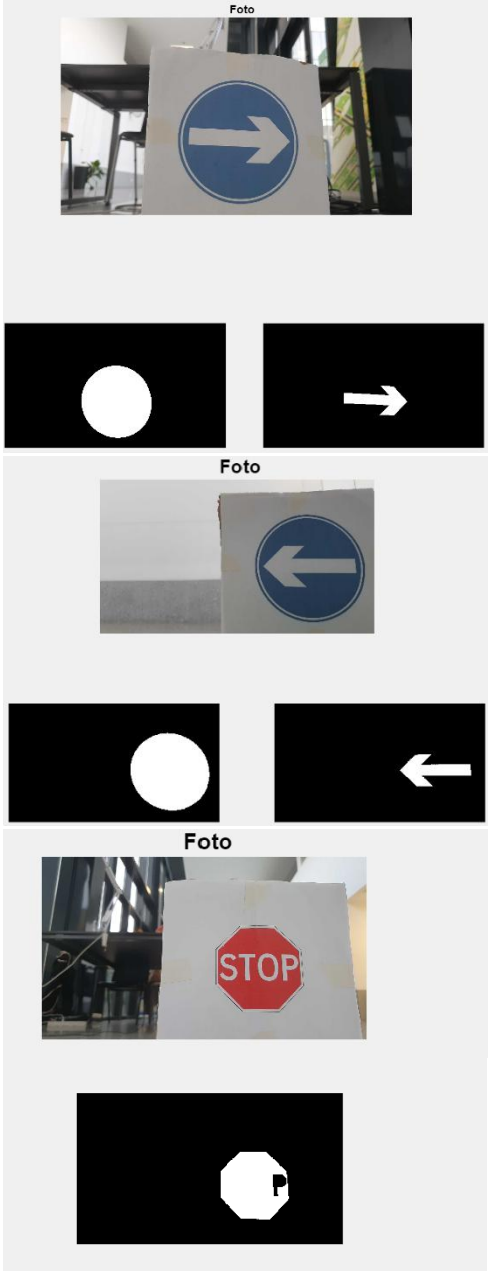


Figura 9 - Reconhecimento de Sinais de Trânsito

A classificação baseada em regras de cor (vermelho para STOP, azul para sinais de obrigação) funcionou de forma confiável. A distinção entre esquerda e direita foi corretamente efetuada através da análise da posição relativa da componente branca (seta) em relação ao centro do sinal.

Foram identificadas algumas situações em que o reconhecimento não funcionou. Estes casos aconteceram principalmente quando o ângulo de visão era muito inclinado ou quando reflexos de luz incidiam diretamente sobre o sinal. Para lidar com estas situações, foi implementada uma lógica que permite ao robô fazer pequenos ajustes na sua orientação antes de tentar novamente a leitura, o que melhorou consideravelmente a robustez do sistema.

5.5 Desempenho Global do Sistema

Durante os testes finais, o sistema conseguiu completar ciclos completos de navegação visitando todas as bases na ordem estabelecida, evitando zonas proibidas e reconhecendo corretamente os sinais de trânsito até encontrar o sinal STOP. O tempo total de execução variou consoante a complexidade da configuração, geralmente variando entre 3 e 5 minutos para configurações com 4 bases.

A taxa de sucesso global situou-se acima dos 85% nos testes realizados. Os principais motivos de falhas foram problemas de iluminação no reconhecimento de sinais ou, mais raramente, erros de posicionamento acumulados que não foram completamente ajustados pela relocalização visual.

5.6 Análise Crítica e Limitações

Embora os resultados tenham sido positivos, foram identificadas algumas limitações que precisam ser discutidas.

A principal vulnerabilidade do sistema de visão é sua sensibilidade às condições de iluminação, o que exige uma calibração cuidadosa dos thresholds HSV para os diferentes ambientes. A segmentação de cores pode se tornar menos confiável em situações com iluminação altamente variável ou sombras acentuadas.

A precisão da odometria, embora aceitável para a maioria dos casos, tem a sua performance comprometida em superfícies irregulares ou quando o robô executa rotações rápidas. A estratégia de relocalização visual mitiga este problema, mas introduz uma dependência da disponibilidade da câmara superior e aumenta o tempo total de execução.

O sistema de planeamento, embora funcional, trabalha com um mapa estático gerado no início da execução. Isto significa que qualquer alteração no ambiente (movimentação acidental de bases, novos obstáculos) não é detetada durante a execução, o que pode levar a comportamentos inesperados. Uma solução futura poderia envolver atualizações periódicas do mapa de obstáculos.

Os resultados obtidos validam a metodologia adotada e demonstram que a combinação dos diferentes módulos permite resolver o problema proposto de forma eficaz. O sistema cumpre os requisitos especificados e as limitações identificadas são esperadas em sistemas desta natureza, podendo ser tratadas através de desenvolvimentos futuros, especialmente no que diz respeito à robustez do sistema de visão e à utilização de técnicas de localização mais avançadas.

6 Conclusão

O objetivo deste trabalho foi desenvolver um sistema para controlo, navegação e visão computacional de um robô móvel, que funciona em um ambiente delimitado com várias bases e zonas proibidas.

A solução desenvolvida configurou-se como sendo robusta e flexível, com destaque na capacidade de mapeamento automático do ambiente a partir de uma única imagem superior, a navegação autónoma entre bases evitando zonas proibidas, e o reconhecimento eficiente dos sinais de trânsito. A implementação da cinemática direta e inversa garantiu um controlo preciso do movimento do robô, enquanto o controlador Drive-to-Point, complementado com mecanismos de desaceleração, zona morta angular e rampa de velocidade, assegurou repetibilidade e estabilidade na execução das trajetórias.

O sistema para planeamento de trajetórias, baseado em mapas de ocupação binários e campos de custo, revelou-se adequado ao problema proposto, possibilitando a criação de caminhos válidos que respeitam as restrições impostas pelas zonas proibidas. A estratégia de relocalização visual contribuiu para a correção dos erros acumulados pela odometria, o que aumentou a precisão do posicionamento do robô junto às bases.

O reconhecimento de sinais através do processamento de imagem no espaço HSV e a classificação através de regras permitiu a correta identificação dos sinais STOP, obrigação à esquerda e obrigação à direita, com robustez face a pequenas variações da iluminação e da perspetiva.

Durante o desenvolvimento, surgiram desafios, nomeadamente a sensibilidade do sistema de visão face às condições de iluminação, a acumulação de erro odométrico ao longo de longas trajetórias, e a necessidade de calibração cuidadosa dos thresholds de cor.

Este trabalho possibilitou a consolidação dos conhecimentos, práticos e teóricos, na área da robótica móvel, visão computacional e sistemas autónomos, demonstrando a aplicabilidade desses conceitos no tratamento de problemas reais de navegação e controlo. O sistema desenvolvido respeita os requisitos especificados e constitui uma boa base para desenvolvimentos futuros.